



Riassunto: Verifica, Testing e Analisi del Software (Capitolo 6)

L'obiettivo di questo capitolo è capire come dimostrare (o meglio, cercare di dimostrare) che il software scritto funzioni correttamente e rispetti le specifiche.

1. I Fondamenti Teorici (Le Basi Assolute)

Il Teorema di Dijkstra (Regola d'Oro)

"Il testing può mostrare la presenza di bug, ma non può mai dimostrarne l'assenza."

Dato che i possibili input di un programma sono infiniti, non esiste la "continuità" nel software. Un programma può funzionare per 1 milione di casi e fallire clamorosamente al milione e uno.

La Catena del Bug (Terminologia Esatta)

All'esame devi usare questi tre termini con precisione ingegneristica:

1. **Error (Errore):** Lo sbaglio umano fatto dal programmatore (es. scrivere $x < 0$ invece di $x \leq 0$).
2. **Fault (Difetto):** Lo stato interno scorretto assunto dal programma in memoria a causa dell'errore (es. la variabile x assume il valore 0 quando non dovrebbe).
3. **Failure (Fallimento):** L'effetto visibile all'esterno. Il programma va in crash o stampa a schermo un risultato palesemente sbagliato.

Il testing cerca i Failure per risalire ai Fault e correggere gli Error.

2. Le Due Famiglie del Testing

A) White-Box Testing (Testing Strutturale)

Il tester **guarda il codice sorgente**. L'obiettivo è coprire il Grafo di Controllo del programma secondo criteri via via più stringenti:

1. **Statement Coverage (Copertura Istruzioni):** Scrivere test in modo che *ogni riga di codice* venga eseguita almeno una volta. È il criterio più debole.
2. **Edge / Branch Coverage (Copertura Rami):** Ogni "freccia" del grafo deve essere percorsa. Per ogni if, devi testare sia il caso in cui è True sia quello in cui è False.
3. **Condition Coverage:** Nelle condizioni composte (es. if (A and B)), devi testare tutte le combinazioni possibili di vero/falso per A e per B singolarmente.
4. **Path Coverage (Copertura Cammini):** Eseguire *tutti* i percorsi possibili dall'inizio alla fine del programma. **È il più forte ma spesso impossibile (Infeasible)** a causa dei cicli while che possono generare percorsi infiniti.

B) Black-Box Testing (Testing Funzionale)

Il tester **non guarda il codice**, ma solo le specifiche.

La tecnica più importante è il **Testing delle Condizioni Limite (Boundary Conditions)**.

- *Perché si fa?* I programmatori sbagliano spesso sui confini delle condizioni.
- *Esempio concreto:* Se la specifica dice "Accetta valori $x > 0$ ", i test Black-Box più importanti da fare sono sul limite: testare $x = -1$, $x = 0$ (per vedere se lo rifiuta correttamente) e $x = 1$ (per vedere se lo accetta).

3. Testing "In The Large" (L'Integrazione)

Dopo aver testato i singoli moduli (Testing in the small), bisogna farli dialogare. Se il Modulo A usa il Modulo B, ma uno dei due non è ancora pronto, usiamo delle "impalcature" (Scaffolding):

- **Stub (Integrazione Top-Down):** È un modulo "fantoccio" vuoto. Viene **chiamato** dal modulo superiore che stai testando. (Es. Testo il modulo CalcolaStipendio, che chiama lo stub LeggiDatabase il quale si limita a restituire un valore fisso senza fare vere query).
- **Driver (Integrazione Bottom-Up):** È un programma "fantoccio". **Chiama** il modulo inferiore che stai testando. (Es. Ho appena finito di scrivere il modulo SalvaSuDisco; scrivo un Driver che fa finta di essere il programma principale e gli inietta dei dati per vedere se salva bene).

4. Analisi Statica e Metriche

Non tutto si verifica facendo girare il codice. Possiamo analizzarlo anche da fermo.

Code Inspection & Walkthrough (Informale)

Piccoli gruppi di lavoro (3-5 persone: autore, moderatore, segretario) che leggono il codice a mente fresca usando delle "Checklist" a caccia di errori tipici (es. array out of bounds, variabili non inizializzate).

Metriche Matematiche

- **Complessità Ciclomantica (Metrica di McCabe):** Calcola il numero di cammini linearmente indipendenti in un modulo.
 - *Formula:* $C = e - n + 2p$ (e = archi, n = nodi, p = componenti connesse).
 - *Regola d'oro:* Un modulo ben scritto e manutenibile deve avere $C \leq 10$. Oltre questo limite, il codice ha troppi if/while ed è un "codice a spaghetti".
- **GQM (Goal - Question - Metric):** Un approccio per misurare la qualità. Prima definisci il *Goal* (es. "Voglio un software veloce"), poi le *Question* (es. "Quanto tempo ci mette a caricare?"), poi la *Metric* (es. "Millisecondi di caricamento"). *Nota bene:* Le metriche non devono **mai** essere usate per valutare/punire i programmatori, ma solo per valutare il

prodotto!



POSSIBILI DOMANDE ALL'ESAME (Con risposte per il 30L)

Domanda 1: "Qual è lo scopo del testing del software?"

Risposta Sbagliata: "Dimostrare che il programma funziona e non ha bug."

Risposta da Lode: "Come affermava Dijkstra, lo scopo del testing non è dimostrare l'assenza di difetti (il che è impossibile dato il dominio infinito degli input), ma evidenziarne la presenza. Il testing è un processo distruttivo: cerchiamo di scatenare dei *Failure* per individuare e rimuovere gli *Error*."

Domanda 2: "Qual è la differenza tra Stub e Driver e quando si usano?"

Risposta da Lode: "Entrambi sono 'scaffolding' usati nell'Integration Testing. Il **Driver** è un modulo finto che *chiama* il componente sotto test, fornendogli input (usato nell'approccio Bottom-Up). Lo **Stub** è un modulo finto *chiamato* dal componente sotto test, che simula la risposta di un componente di basso livello non ancora implementato (usato nell'approccio Top-Down)."

Domanda 3: "Perché la Path Coverage (Copertura dei cammini) è considerata il criterio più forte ma spesso inapplicabile?"

Risposta da Lode: "La Path Coverage richiede l'esecuzione di ogni singola combinazione di percorsi dal nodo iniziale al nodo finale del grafo di controllo. È spesso inapplicabile (infeasible) a causa della presenza dei cicli (es. while). Un ciclo dipendente da un input utente può generare un numero teoricamente infinito di cammini diversi, rendendo impossibile testarli tutti in tempo finito."

Domanda 4: "Mi definisca la metrica di McCabe e mi dica qual è il suo limite consigliato."

Risposta da Lode: "È la Complessità Ciclomatica, si calcola sul grafo di controllo con la formula

$$C = e - n + 2p$$
 (archi - nodi + 2*componenti connesse). Misura la quantità logica di decisioni (diramazioni) presenti nel codice. McCabe suggerisce che un singolo modulo

dovrebbe avere una complessità massima di ¹⁰; oltre questa soglia, il modulo diventa troppo difficile da testare esaustivamente e da mantenere, e andrebbe frammentato."

Domanda 5: "Qual è la differenza tra Error, Fault e Failure?"

Risposta da Lode: "Sono la catena di causa-effetto di un difetto. L'*Error* è l'azione umana scorretta (es. il programmatore fa un errore logico nel codice). Il *Fault* è la manifestazione dell'errore nel software (es. una variabile che assume uno stato scorretto in memoria). Il *Failure* è l'evento osservabile in cui il sistema viola le specifiche e fallisce visibilmente durante

l'esecuzione."